Practical – 8

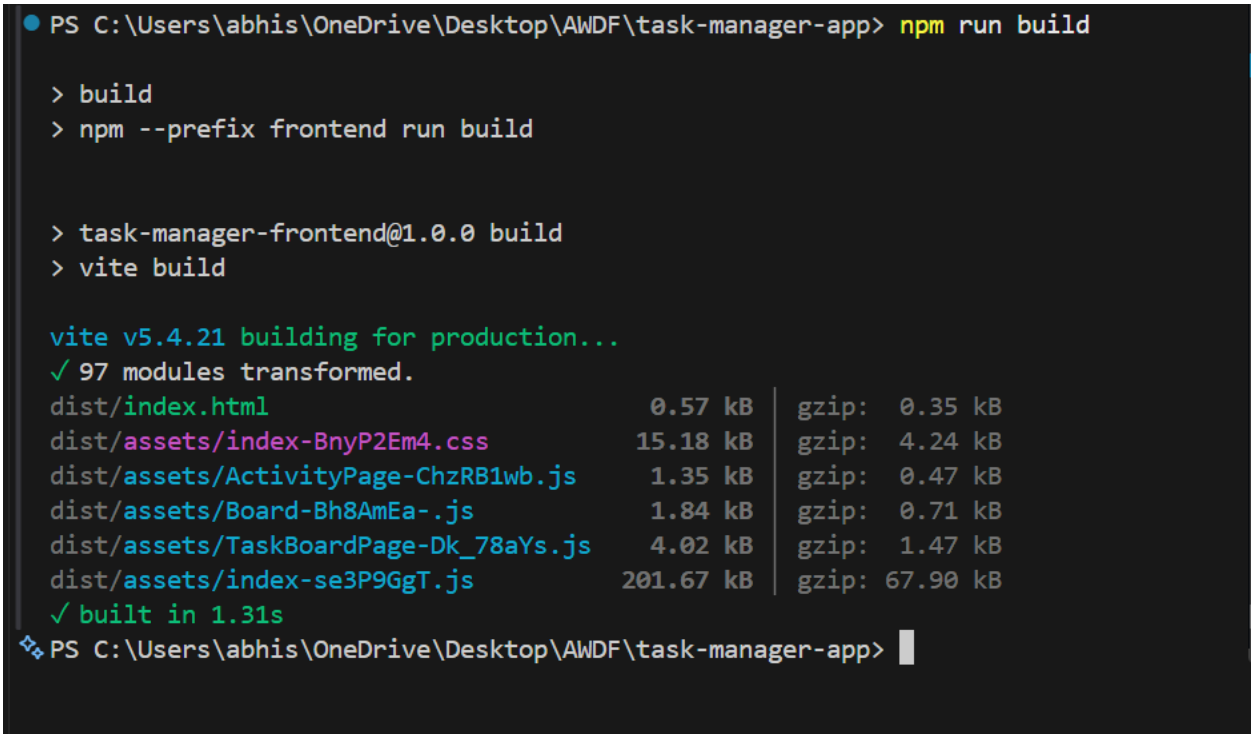
Aim:

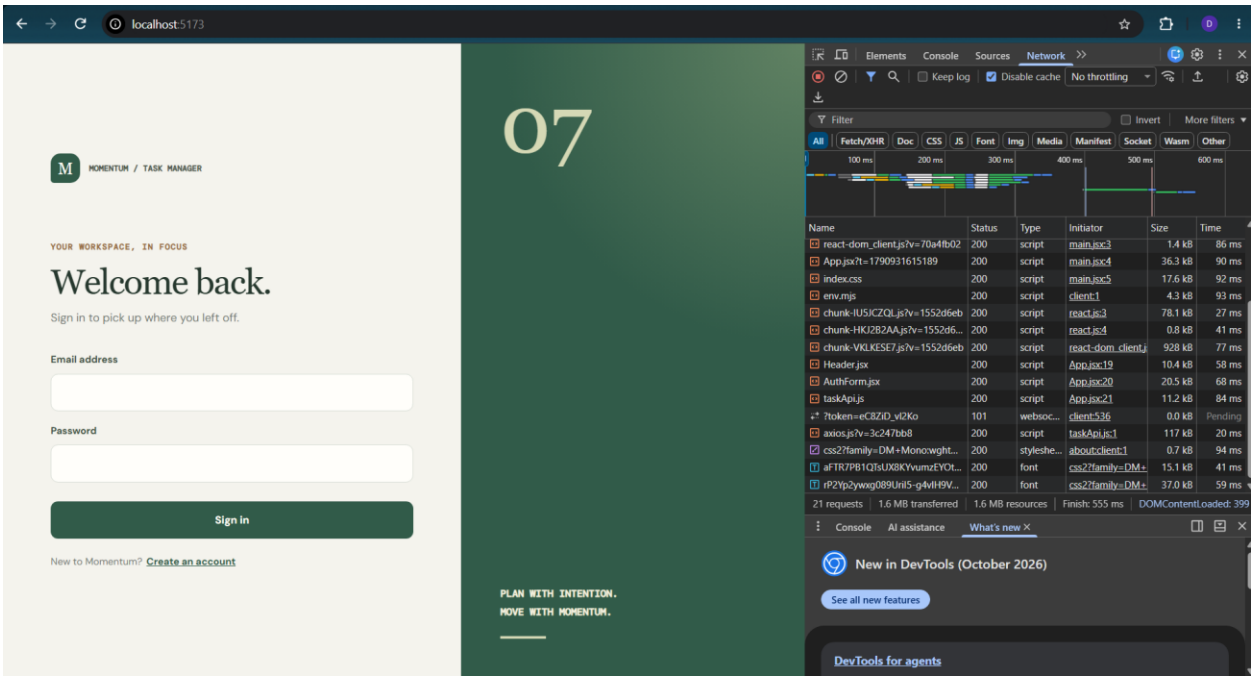
To improve frontend performance using lazy loading and code splitting techniques.

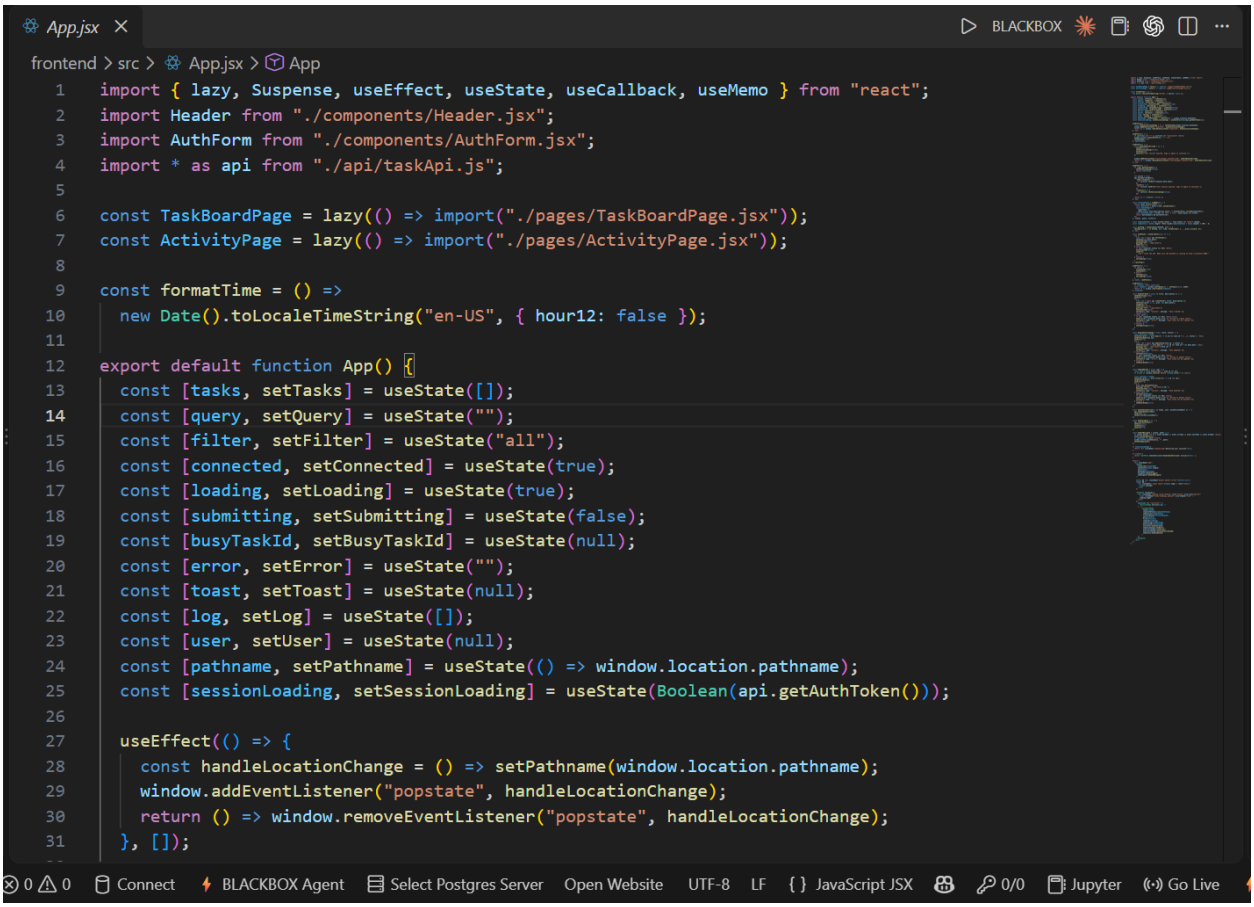
Content:

After completing this practical, I will be able to optimize the performance of a React-based Task Management application using lazy loading and code splitting. I will learn to implement React.lazy() and Suspense, load route-based components on demand, display loading fallback UI, and measure performance using browser developer tools. This practical will help me reduce the initial bundle size, improve loading performance, and develop efficient and responsive React applications.

Screen Shot:



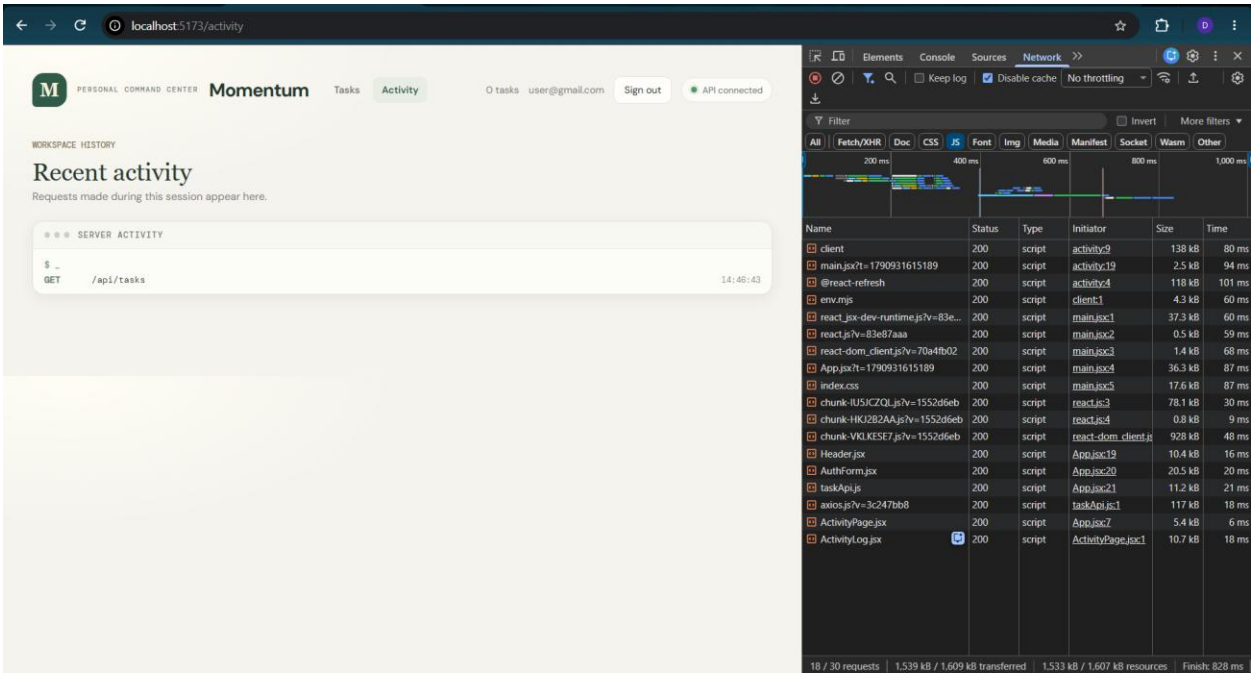

localhost:5173
MOMENTUM / TASK MANAGER
YOUR WORKSPACE, IN FOCUS
Welcome back.
Sign in to pick up where you left off.
Email address
Password
Sign in
New to Momentum? Create an account
07
PLAN WITH INTENTION.
MOVE WITH MOMENTUM.
Elements
Console
Sources
Network
New in DevTools (October 2026)
See all new features
DevTools for agents

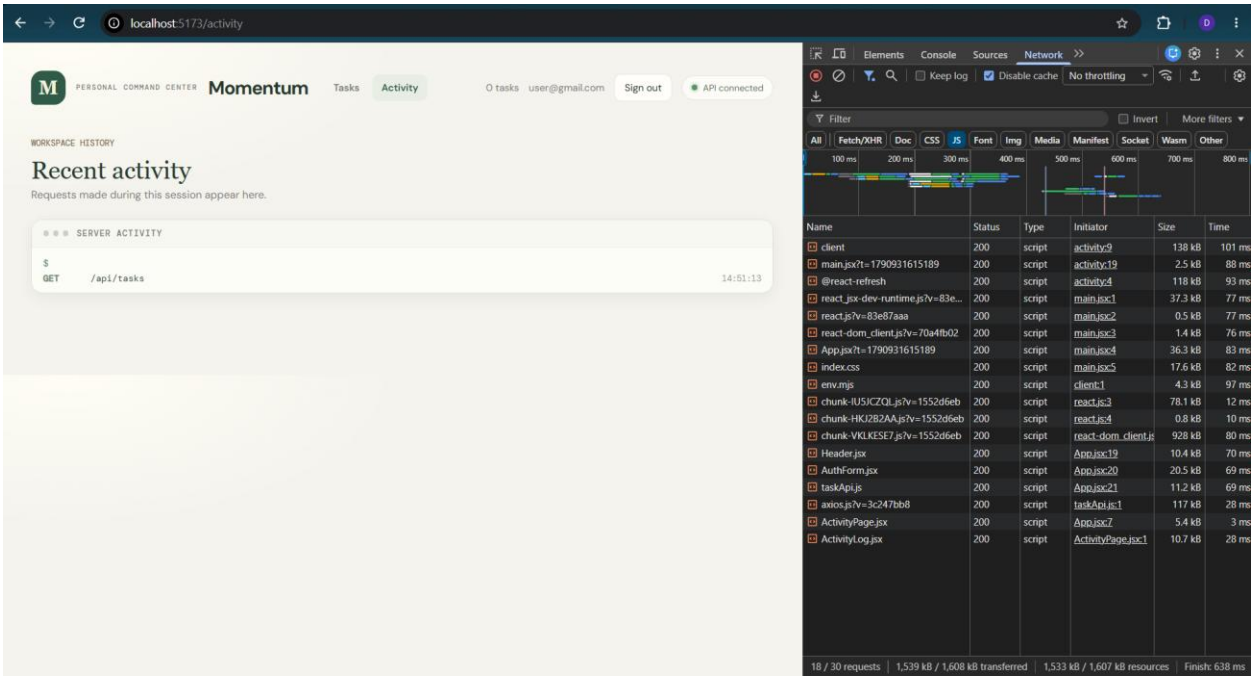

App.jsx
BLACKBOX
frontend > src > App.jsx > App
import { lazy, Suspense, useEffect, useState, useCallback, useMemo } from "react";
import Header from "./components/Header.jsx";
import AuthForm from "./components/AuthForm.jsx";
import * as api from "./api/taskApi.js";
const TaskBoardPage = lazy(() => import("./pages/TaskBoardPage.jsx"));
const ActivityPage = lazy(() => import("./pages/ActivityPage.jsx"));
const formatTime = () =>
new Date().toLocaleTimeString("en-US", { hour12: false });
export default function App() {
const [tasks, setTasks] = useState([]);
const [query, setQuery] = useState("");
const [filter, setFilter] = useState("all");
const [connected, setConnected] = useState(true);
const [loading, setLoading] = useState(true);
const [submitting, setSubmitting] = useState(false);
const [busyTaskId, setBusyTaskId] = useState(null);
const [error, setError] = useState("");
const [toast, setToast] = useState(null);
const [log, setLog] = useState([]);
const [user, setUser] = useState(null);
const [pathname, setPathname] = useState(() => window.location.pathname);
const [sessionLoading, setSessionLoading] = useState(Boolean(api.getAuthToken()));
useEffect(() => {
const handleLocationChange = () => setPathname(window.location.pathname);
window.addEventListener("popstate", handleLocationChange);
return () => window.removeEventListener("popstate", handleLocationChange);
}, []);
Connect
BLACKBOX Agent
Select Postgres Server
Open Website
UTF-8
LF
JavaScript JSX
Jupyter
Go Live

App.jsx

frontend > src > App.jsx > App

```jsx
export default function App() {
  <Suspense fallback={(
    <div className="loading route-loading" role="status" aria-live="polite">
      <span className="route-loading__spinner" aria-hidden="true" />
      Loading page…
    </div>
  )}>
    {pathname === "/activity" ? (
      <ActivityPage entries={log} />
    ) : (
      <TaskBoardPage
        tasks={tasks}
        completedTasks={completedTasks}
        completion={completion}
        filteredTasks={filteredTasks}
        query={query}
        filter={filter}
        loading={loading}
        submitting={submitting}
        busyTaskId={busyTaskId}
        onCreate={handleCreate}
        onQueryChange={setQuery}
        onFilterChange={setFilter}
        onStatusChange={handleStatusChange}
        onDelete={handleDelete}
      />
    )}
  </Suspense>
</div>
);
```





Conclusion:

This practical demonstrates the implementation of JWT-based authentication and input validation in a full-stack Task Management application using Node.js, Express.js, MongoDB, and Mongoose. It includes user registration and login functionality, secure password hashing using bcrypt, JWT token generation and verification, and protected routes using authentication middleware. It also covers server-side input validation, error handling, and testing of authentication and authorization through Postman.